

Paddle Inference源码阅读

目录

- 背景
- 源码阅读目标
- Native模式
 - 3.1 预测的基本数据结构
 - 3.2 Load模型
 - 3.3 模型预测
- Analysis模式
 - 4.1 初始化Predictor
 - 4.2 Analysis passes处理流程

 本页面由方泽阳于2022/04/06迁移自wiki 方泽阳空间

背景

目前模型预测这块主要是有两种模式(在paddle中不同的预测方式以及预测配置称为不同的预测模式)，第一种是native模式(原生的paddle的预测模式)，第二种是analysis模式(做了图计算优化和底层OP优化,包含在不同硬件下汇编级别的加速)，Analysis模式中可以调用两种engine，一种是TensorRT engine，tensortrt机制是一种GPU的硬件加速机制，对于CNN和GRU这种的网络结构具备很好的加速预测能力。第二种是Anakin engine（目前是公司内部维护的深度学习预测推理框架，主要是将不同的模型进行输入然后通过Anakin parser进行解析成统一的网络图结构，供图优化和底层硬件加速）。

源码阅读目标

- 1) native模式
- 2) analysis模式下analysis engine
- 3) analysis模式下tensorrt engine

Native模式

3.1 预测的基本数据结构

PaddleBuf来构造数据，一种初始化方式设置PaddleBuf设置length，第二种是传入data和length进行数据的构造。

使用PaddleTensor来作为模型的输入输出，具体的定义如下：

PaddleTensor定义

```
</> C++ | 收起 ^  
  
1 struct PaddleTensor {  
2     PaddleTensor() = default;  
3     std::string name; // variable name.  
4     std::vector<int> shape;  
5     PaddleBuf data; // blob of data.  
6     PaddleDType dtype;  
7     std::vector<std::vector<size_t>> lod; // Tensor+LoD equals LoDTensor  
8 };
```

3.2 Load模型

- 通过paddle::inference::Load接口将模型的路径输入进去，由于模型文件是二进制文件，同时通过导入文件来初始化ProgramDesc这个类，这个类主要是存储了模型相关的信息，同时可以对模型的输入输出进行操作。通过将二进制文件转为Proto文件，这样就可以利用void ProgramDesc::InitFromProto()函数将Proto文件来还原所有的Block信息。
- 通过Executor::Prepare函数将block种关于的op的名字通过接口将op名注册成OP类，供后续调用。OP注册后的类信息时放入到ExecutorPrepareContext种存储。
- Executor::CreateVariable函数主要是将programe_desc中根据Variable的信息来注册成Variable类，供预测使用，Variable信息放入到Scope存储。对于Persistable的变量则是需要通过对模型的参数的解析进行读取，主要的方式是对读取的数据的开启一个子program然后增加一个load的OP操作，这样就可以把数据load到到scope里面去了。（对于模型参数中可变的和非可变的，可变的部分基本上模型预测过程的中间变量，而非可变部分是固定参数部分，因此在scope的处理上稍有不同，对于固定参数部分存在scope里面，对于可变部分的参数则会在scope下面新建一个sub_score,这样的设计的好处在如果有多线程预测，scope中固定参数部分可以被多线程共享，不需要进行深拷贝，而可变部分根据需要创建，线程之间相互独立）
- NativePaddlePredictor::PrepareFeedFetch设置fetch和feed的OP操作信息。

3.3 模型预测

基本就是执行executor_→RunPreparedContext操作，和训练模型的基本一致，基本上按照已经解析好的网络结构，顺序的执行的Block中的OP操作，对需要的Input和Output分别从scope中

提取和存储。

Analysis模式

Analysis模式主要是通过优化优化计算图和底层计算OP的优化，来加速预测的速度，主要是支持MKL-DNN(Intel开源的基于CPU深度学习加速的框架)和TensorRT(英伟达开源的基于GPU深度学习加速的框架)，可以使用一下的命令来看CPU的型号，从而判断是否可以CPU加速；同时在Analysis engine中是有选项是否进行Anakin加速的。

</>

Plain Text | 收起 ^

```
1 cat /proc/cpuinfo | grep 'model name' | uniq
```

analysis engine的使用方式和Native engine很类似，基本上都是初始化predictor，load模型已经模型预测，具体源码分析如下。如果需要Analysis engine需要了解一下几个概念。IR (intermediate representation) 中间表达式，在深度学习中涉及到不同的框架的通用问题，深度学习IR就是希望A框架->IR->B框架，目前开源比较火的开源框架有ONNX[链接](#)。

4.1 初始化Predictor

AnalysisPredictor::PrepareScope初始化scope和sub_scope,并且设置当前状态为非Clone状态(这个时候可以设置MKL-DNN加速的线程数)；创建Executor；

AnalysisPredictor::PrepareProgram根据模型参数和结构来创建Program，主要是创建Block，同时赋值Persistable的参数。同时AnalysisPredictor::PrepareProgram在创建Program做了一些优化，主要优化地点主要是基于不同优化工具的计算图优化，具体的有以下信息。Analysis模式的基本处理流程如下：

- 1) 将Analysis预测模式划分为不同的阶段，代码中定义为Aanalysis passes,不同的engine有不同的执行的passes，例如Anakin engine，基本上分别分为infer_clean_graph_pass（建立不同数据节点中间的先后操作问题），ir_builder_passes(主要是将proto的网络结构转为图结构的网络结构，然后对图结构的网络结构进行图优化和OP操作优化)，ir_params_sync_among_devices_pass（如果在不同的设备上计算，需要在不同的设备上同步预测配置），ir_graph_to_program_pass（将图网络结构再转化为Proto的网络结构，供Executor调用）
- 2) ir_builder_passes 主要是将Proto的网络结构转化为Graph的网络结构，转为一种中间态的网络结构，IR是中间表达式的意思，在中间表达式做OP操作优化，例如不同OP操作的融合，这个过程不同的Engine有不同的自定义过程和实现。

Analysis Configure配置

- GPU:使用TensorRT来进行计算图优化
- CPU:使用MKL-DNN来优化底层OP操作

- 打开Anakin-使用Anakin来进行图计算优化，可以适配于CPU和GRU
- MKL-DNN可以做优化的OP列表 这个需要加进去（需要手动加入配置）
- MKL-DNN进行图像量化操作列表(需要手动设置)
- 设置Analysis的构图的phrase阶段
- 设置Analysis的IR-passes的阶段（定义图优化的具体的passes）

4.2 Analysis passes处理流程

4.2.1 定义Analysis Pass builder

定义Analysis Pass builder，pass builder里面设置可以在不同的设备下建立不同的pass builder，pass builder可以设置需要解析的passes，例如先将Fluid版本的模型的转化为Graph结构，再对Graph进行图优化，在图优化的过程中进行OP的性能优化。

```
</> C++ | 收起 ^  
  
1 config_.pass_builder()  
2 //pass_builder里面去根据CPU和GPU的情况来调用不同的builder来增加不同的IR-PASSES  
3 CpuPassStrategy::CpuPassStrategy() : PassStrategy({}) {  
4     // NOTE the large fusions should be located in the front, so that they  
5     // will  
6     // not be damaged by smaller ones.  
7     passes_.assign({  
8         "infer_clean_graph_pass",          //  
9         "attention_lstm_fuse_pass",        //  
10        "seqpool_concat_fuse_pass",        //  
11 //.....此处省略.....
```

4.2.2 执行Analysis pass builder

```
</> C++ | 收起 ^  
  
1 Analyzer().Run(&argument_);  
2 //这里已经加入到参数列表中的passes结合开始执行  
3 //这里会对会分析Pass信息，具体分析pass有 ir_graph_build_pass  
4   ir_analysis_pass   ir_params_sync_among_devices_pass  
5 void Analyzer::RunAnalysis(Argument *argument) {  
6     PADDLE_ENFORCE(argument->analysis_passes_valid(),  
7         "analsis_passes is not valid in the argument.");  
8     for (auto &pass : argument->analysis_passes()) {  
9         string::PrettyLogH1("--- Running analysis [%s]", pass);  
10        auto *ptr = PassRegistry::Global().Retreive(pass);  
11        PADDLE_ENFORCE_NOT_NULL(ptr, "no analysis pass called %s", pass);
```

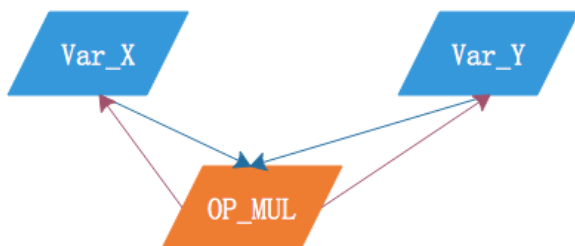
```

11     ptr->Run(argument);
12 }
13 }

```

• ir_graph_build_pass

主要工作是通过对模型的结构进行读取，然后转化为Graph结构，在图结构里面是有Node结构组成，图节点分为数据节点和操作节点。



操作节点的输入是数据节点
数据节点的输出是操作节点

通过Load模型结构最后来进行Graph构造，但是只通过数据节点来构成Graph，解决数据节点的读写先后问题，通过Hazard算法解决读写依赖的问题。

</>

Java | 收起 ^

```

1 node->inputs.push_back(var); //var数据节点 node是操作节点
2 var->outputs.push_back(node)
3 /*
4 如何解决读写依赖的问题，主要是插入空OP和空var的方式，让读写之间建立依赖
5 var->inputs(write_op)
6 var->outputs(read_op)
7 对于WAW 需要建立一个CreateControlDepVar空依赖的Var，让VAR的输入是前面的Write操作
  (w_stage1)，输出是后面的Write操作 (w_stage2)
8 对于WAR 如果两者之间已经有Var的依赖那就不需要建立依赖的Var节点，如果两者没有依赖的
  Var，那就不需要建立一个空依赖数据Var，输入是读操作，输出是写操作
9 */

```

• ir_analysis_pass

在ir_graph_build_pass阶段将fluid的PB模型结构转化为了网络图结构，而在ir_analysis_pass阶段主要是根据不同的engine和配置来对IR-passes进行解析，对Graph类的图网络结构进行优化以及OP方面的优化。下面主要是看一下Anakin的passes的执行过程，来观察优化过程。

</>

C++ | 收起 ^

```

1 IRPassManager the_ir_manager(argument); //这里主要对IR-passes阶段的不同的
   pass填入参数
2 graph = the_ir_manager.Apply(std::move(graph)); //执行各个passes
3 //下面是passes是Anakin的执行过程
4 const std::vector<std::string> kAnakinSubgraphPasses({
5     "infer_clean_graph_pass",           //
6     "quant_conv2d_dequant_fuse_pass",   //
7     "simplify_anakin_priorbox_detection_out_pass", //
8     "fillconstant_elementwisemul_fuse", //
9     "fc_fuse_pass",                     //
10    "conv_elementwise_add_fuse_pass",    //
11    "fc_gru_fuse_pass",                  //
12    "shuffle_channel_detect_pass",       //
13    "anakin_subgraph_pass",              //
14    "fc_gru_fuse_pass",                  //
15 });

```

1) IR-pass infer_clean_graph_pass 阶段

对于构造模型网络的graph的过程中的无效的依赖Var节点进行清除（会保存原始的图结构）

2) IR-pass quant_conv2d_dequant_fuse_pass

这里主要是优化inference图结构，如果遇到fc conv2d的这样的一些OP操作的时候，并且这些op操作都是使用量化和反量化的时，在inference的时候也会进行图优化，让这些OP融合在一起。

</>

C++ | 收起 ^

```

1 //创建一个PDNode节点，这个节点会增加会增加各种匹配Node中OP的函数
2 //例如我们想判断一个数据节点的下一步输出到哪些OP里面去就可以通过PDNode的assert函数去
   增加匹配函数
3 auto* x = gpd.mutable_pattern()
4         ->NewNode("x")
5         ->assert_is_op_input(quant_type, "X")
6         ->AsInput();
7 //这里只会定义PDNode开始的节点，存在GraphPatternDetector里面
8 //同时会构造一个专属的Pattern，里面会一定整合OP的一系列pattern，同时用edge将这些
   pattern连接起来，具体可以参考例子PDNode
   *patterns::ConvElementwiseadd::operator()
9 //通过GraphPatternDetector这个解释器在Graphe里面去查找满足pattern的子图并且返回
10 std::vector<GraphPatternDetector::subgraph_t>
11 GraphPatternDetector::DetectPatterns();
12 //operator()函数里面主要是做了一下的事情

```

```

13 //1.从整个图里面找到满足pattern的IRNode,建立PNode->IRNode的关系 (查找方法是通过深度
    优先的方式来查找)
14 bool GraphPatternDetector::MarkPDNodesInGraph
15 //2.构建子图, subgraph_t = std::unordered_map<PDNode*, Node*>; 查找子图就是
    操作对应的PDNode
16 //构建子图的方法主要是通过PDNode方式的关系来进行DFS的查找,如果edge的两端在原始的
    Node的关系也是存在的那就确认可以匹配到两个节点,同时利用算法查找最大的子图
17 for (auto *node : pdnodes2nodes_[first_pnode]) {
18     HitGroup group;
19     group.roles[first_pnode] = node;
20     init_groups.emplace_back(group);
21 }
22 int step = 0;
23 bi_records[0] = std::move(init_groups);
24 for (const auto &edge : pattern_.edges()) {
25     VLOG(4) << "check " << edge.first->name() << " -> " << edge.second-
        >name();
26     auto &pre_groups = bi_records[step % 2];
27     auto &cur_groups = bi_records[1 - (step++ % 2)];
28     cur_groups.clear();
29     if (pre_groups.empty()) break;
30     for (Node *source : pdnodes2nodes_[edge.first]) {
31         for (Node *target : pdnodes2nodes_[edge.second]) {
32             VLOG(8) << "check " << source->id() << " -- " << target->id();
33             // TODO(Superjomn) add some prune strategies.
34             for (const auto &group : pre_groups) {
35                 if (IsNodesLink(source, target)) {
36                     HitGroup new_group = group;
37                     bool flag = new_group.Match(source, edge.first) &&
38                                 new_group.Match(target, edge.second);
39                     if (flag) {
40                         new_group.Register(source, edge.first);
41                         new_group.Register(target, edge.second);
42                         cur_groups.push_back(new_group);
43                         // TODO(Superjomn) need to unique
44                     }
45                 }
46             }
47         }
48     }
49 }
50 //3.构造好子图后要进去unique处理 A * B + bais -> C B * A + bais -> C 这种操作
    是一样的没有必要构造成两个子图
51 /*4.最后调用handler处理

```



```

52         auto handler = [&](const GraphPatternDetector::subgraph_t&
subgraph,
53                             Graph* g);
54 handler的主要功能是构造的子图，根据子图里面的数据节点构造一个新的OP，因此来进行图计
算优化
55 */

```

3) IR-pass anakin_subgraph_pass

目前Paddle Fluid的版本是可以兼容Anakin的engine，Anakin框架的核心逻辑如图1所示，主要由Parser, Framework 和Saber组成。Parser是独立解析器，用于将不同训练框架生成的模型转化为统一的Anakin图描述。Framework是框架主体，使用C++实现，用于完成硬件无关的所有操作，比如构建网络、图融合、资源复用、计算调度等。Saber是一个高效的跨平台计算库，包括大量的汇编级优化代码，并支持众多国际行业合作伙伴的架构，如Intel-cpu，NV-gpu，AMD-gpu和ARM。

</>

C++ | 收起 ^

```

1 //anakin_subgraph_pass主要是将已经解析好的子图中的Graph中某些OP进行替换，替换成
Anakin支持的OP，因为Anakin的OP可以进行部分量化处理，同时在底层进行了汇编级别的优化
2 analysis::AnakinSubgraphPass::ApplyImpl
3 //首先check哪些OP可以支持优化同时将这些OP节点打上标记，同时将于此相关的Var节点也要打
上标记
4 //node_inside_subgraph_teller_表示匹配函数
5 //在首先使用算法寻找子图
6 SubgraphDetector::ExtractSubGraphs() //查找子图的入口函数
7 //使用的是拓扑排序的方式来遍历所有的节点，先判断入度为0的节点，然后保存起来，然后入
口入度为0的节点从图中删除，同时把和这个节点相连的所有其它的节点，入度减1
8 //循环遍历
9 framework::ir::GraphTraits::TS //拓扑排序入口 结合拓扑排序和Graph来查找满足接口
的sub-graph
10 //找到子图后主要是有两步操作，第一步设置一个OP-Node，这个操作节点包含子图，同时设置
这个节点的输入和输出；
11 //第二步，是将子图的节点从原始的graph里面删除掉，然后将这个节点与剩余的其它节点相连，
构造一个新的子图。
12 //设置输入输出主要方法是如果这个节点不在子图里面，但是和子图相连，那么就是输入和输出
13 //ExtractInputAndOutputOfSubGraph(std::vector<Node *> &graph) 查找子图的
函数
14 auto inlink_in_subgraph = [&](Node *n) {
15     for (auto *in : n->inputs) {
16         if (nodes.count(in)) return true;
17     }
18     return false;
19 };
20 ExtractInputAndOutputOfSubGraph(std::vector<Node *> &graph)

```



```

21  for (auto &node : graph) {
22      for (auto *in : node->inputs) {
23          // The Value that is written by nodes inside a sub-graph shouldn't
          be the
24          // input of the sub-graph.
25          if (!nodes.count(in) && in->IsVar() && !inlink_in_subgraph(in)) {
26              inputs.insert(in);
27          }
28      }
29      for (auto *out : node->outputs) {
30          if (!nodes.count(out) && out->IsVar()) {
31              outputs.insert(out);
32          }
33      }
34  }
35  ////上诉的方法有个问题是如果输出是中间的结果的输出，也会放入作为新加的OP节点的输出里面
    去，所以要去掉
36  RemoveIntermediateOutputInSubgraph
37  //然后将子图中的Node节点设置为delete状态，同时将子图节点与其它剩余节点相连
38  //这里就是子图节点中如果有有一些中间节点，这个也可以删除地掉
39  FilterRedundantOutputOfSubGraph

```

结合上诉的方法就可以找到子图，并且通过一个新OP节点替换原始的子图，下面就是如何在新的OP节点上面进行OP优化，主要是通过将native的OP节点转化为Anakin版本的OP节点

</>

C++ | 收起 ^

```

1  //将原始的操作Node转化为Anakin 操作节点函数
2  void AnakinSubgraphPass::CreateAnakinOp(
3      framework::ir::Node *node, Graph *graph,
4      const std::vector<std::string> &graph_params,
5      std::vector<std::string> *repetitive_params);
6  //先加sub-graph中node节点新增到一个Block里面去，作为一个sub_block
7  for (auto *node : subgraph) {
8      auto *new_block_op = new_block->AppendOp();
9      auto *op = block_desc.AppendOp();
10     *new_block_op->Proto() = *node->Op()->Proto();
11     *op->Proto() = *node->Op()->Proto();
12 }
13 //设置新OP的参数信息，例如输入输出，OP操作名，是否使用gpu等参数
14 //在设置好block信息后，以及Anakin需要的参数信息，开始创建Anakin engine
15 //这里会做Int8的类型的区分
16 if (enable_int8) {
17     CreateAnakinEngine<::anakin::Precision::INT8>(&block_desc, params,

```

```
18                                     input_names,
    output_mapping,
19                                     program_inputs,
    engine_key);
20 } else {
21     CreateAnakinEngine<::anakin::Precision::FP32>(&block_desc, params,
22                                     input_names,
    output_mapping,
23                                     program_inputs,
    engine_key);
24 }
25
26 //CreateAnakinEngine函数里面主要是将输入的block里面的信息开始解析
27 //将block里面的OP转化为Anakin的op
28 ConvertBlock(block_desc, parameters, *scope, engine);
29 //设置这个Block的输入输出，把它转化为Anakin需要的数据格式
30 //最后Anakin engine会调用接口进行组网和优化
31 engine->Optimize();
32 engine->InitNet();
```

- [ir_params_sync_among_devices_pass](#)

如果同时在CPU和GPU上设备上都开启了优化操作，则需要把某个设备上的对参数（Scope上的内容）要同步到另外一个设备上。

- [ir_graph_to_program_pass](#)

将graph的网络结构转化Progame的OP的PB结构供Executor调用